SDK Audit: Work Split (Habiba & Ehsan)

Status: **FINAL**, 2026-10-07. Paths, `file:line` citations, line counts and imports between your two areas were checked against the repo.

---

0. In one minute

- Two people, two areas, so you rarely edit the same files: **Habiba** = Python backend services + database. **Ehsan** = LiveKit worker + npm SDKs + dashboard + build/release. Section 2 shows each person's modules at a glance.
- 12 audit modules (M1 to M12). Each has a lead, sometimes a second person who reviews the part that touches their code.
- Do **P0** modules first (auth, tenant isolation, money), then P1, then P2.
- `file:line` references are **pointers**. Open the file yourself before you write a finding.
- Section 7 lists code nobody has read yet (with line counts). Several modules depend on it.
- Findings use the template in Section 9.

---

1. Constraints we audit under

Dashboard is read-only (Habiba's definition): users can *read* everything in the dashboard, including agents that were created in code (e.g. via the agents SDK), but users cannot *create agents from the dashboard*. Whether edits, credential reveal/rotate, telephony actions and Test Studio also count as "read-only" is **not decided yet** (see M11 and Section 8).

LiveKit worker runs locally for now. One machine, not hosted.

Other backend services run on Render's free plan.

What that implies (to confirm, not conclusions)

Render and LiveKit points come from vendor docs and third-party summaries as of Oct 2026. Confirm them in your own Render dashboard.

- **Render free:** services spin down after about 15 minutes idle and take about a minute to wake. They can restart at any time, the filesystem is not persistent, instance hours are shared across the workspace, and services can't scale out. So anything held in memory is lost on restart: rate-limit buckets, webhook de-duplication caches, demo caches. Client timeouts also matter: the voice SDK defaults to 15 s, the worker's tool calls have a 4 s read timeout, and the agents/telephony SDKs set no timeout.
- **Local worker:** capacity is one machine's CPU, RAM and network. LiveKit's default load check is CPU-based and a worker stops taking new jobs above a threshold. A sleeping laptop or lost network ends live calls. The machine also holds production-grade secrets (see M7). "Hosted vs local" is decided in code: `admin/app.py:L65-L70` reads `UVA_ENV` / `RENDER`, `worker/recording_policy.py:L40` reads `RENDER` only, and the worker's SSRF guard asks `control_plane.runtime_env.is_hosted` (`worker/ssrf_guard.py:L15`). A local worker will probably look "not hosted" to these checks; see M6.
- **Dashboard read-only:** hiding buttons is not enough. It has to be enforced in the portal API and in the database (RLS), because the dashboard also holds a Supabase browser client (`dashboard/src/lib/supabaseBrowser.ts:L19-L25`).

Confidence note

The facts in this plan come from a first read of the code by an AI assistant. Paths, `file:line` citations, line counts and imports between your two areas were then re-checked against the repo, but other claims are not independently verified. No dependency-graph tool was available, so coupling comes from imports and grep only.

---

2. Who does what

Habiba takes the Python backend services and the database. **Ehsan** takes the worker, the npm SDKs, the dashboard, and build/release.

	Habiba: backend gatekeepers	Ehsan: runtime, client surface, build & release
Leads	M1, M2, M4, M5, M8	M3, M6, M9, M10
Shares with the other	M7 (server side), M11 (server and database enforcement), M12 (Python tests)	M7 (worker and laptop), M11 (UI and data visibility), M12 (npm and worker tests)
Reviews for the other	M6: the portal's save-time tool checks	M1: client signers. M2: worker release and SDK refresh. M4: telephony SDK. M8: SDK and demo timeouts
Code	`control_plane/` (2,167 LOC), `tenant_portal_api/` (12,502 across 32 files; 12,272 without `test_studio.py`), `admin/` (1,117), `supabase/migrations/**` (39 SQL files, unread)	`worker/` (10,476), npm SDKs `sdk/` `sdk-server/` `telephony/` (2,120), `dashboard/` (8,620 `.ts/.tsx`, 70 files), demos/starters (2,811), `host_tools/` (264)

	`tions/**` (39 SQL files, unread)	(2,811), `host-tools/` (364)
Also owns	database migrations; DB/tenant/telephony ops scripts (below)	`.github/workflows/**`, `docker/**`, `Makefile`, `requirements*.txt`, root `package.json` / `pnpm-lock.yaml`, `.env.example`; worker/provider scripts (below)
Approx size	~15.8k LOC + migrations	~24.4k LOC (dashboard and demos are lower-depth)
Why this cut	`tenant_portal_api` is one package: one owner avoids merge conflicts. Holds most P0 risk (auth, tenants, telephony, quotas).	Everything that runs the call, ships to developers, or builds and releases.

Shared-file rule: **Habiba owns the database and migrations. Ehsan owns build and release files.** The other one asks before editing.

Rebalance rule: if one of you falls behind, the other takes the lowest-priority modules first (M12, then the demo parts of M9).

Code that is in no module: decisions

Path	Decision
`scripts/**` (50 files)	Owned by what they touch (assigned by filename; confirm by imports). **Habiba:** `migrate.py`, `db_inspect.py`, `db_reset.py`, `dbconn.py`, `rls_check.py`, `encrypt_tenant_secrets.py`, `purge_used_nonces.py`, `purge_expired_session_media.py`, `reconcile_sessions.py`, `reconcile_telephony.py`, `usage_guard.py`, `verify_admin_boundary_live.py`, `verify_rate_limit_live.py`, `provision_admin.py`, `provision_demo_tenant.py`, `create_new_client.py`, `mint_demo_token.py`, the Telnyx `*.mjs` helpers. **Ehsan:** `dev_worker.py`, `drain_stale_agent_jobs.py`, `simulate_worker_crash.py`, `concurrency_test.py` (+ `.html`), `measure_gemini_tpm.py`, `ponytail_measure.py`, `probe_soniox_402.py`, `provision_*_test_agent.py`, `record_*`, `fetch_*_voices.py`, `upload_voice_previews.py`, `resign_voice_previews.py`, `update_phrase_config.py`, `backfill_english_voice_gender.py`, `fixture_stats.py`, `run_ci_unit.py`, `gate0.sh`, `assemble_phase8_release_candidate.py`. Existing safety tools worth reading and running early: `rls_check.py`, `verify_rate_limit_live.py`, `verify_admin_boundary_live.py` (Habiba) and `concurrency_test.py`, `simulate_worker_crash.py`, `drain_stale_agent_jobs.py` (Ehsan).
`tests/**` (128 files)	The owner of the code under test. M12 inventories them.
`docs/**`	Only `docs/HOST_BACKEND_CONTRACT.md` is in scope (M9). The rest is unread and has no module.
`services/` (63 LOC, TTS fixture cache)	Not audited.
`self-serve-demo-UI/`, `voice-picker/`, `state/`	Not assessed. `self-serve-demo-UI/` contains its own npm packages (see Section 8, Q9).
`.codex/`, caches, `.venv/`	Ignore.

Suggested order

- **Habiba:** M1, M5, M4, M2, M8, M11 (enforcement), M7 (server side), M6 (review of save-time checks), M12 (Python)
- **Ehsan:** M3, M2 (worker side), M6, M9, M10, M11 (UI/visibility), M7 (worker/laptop), M12 (npm/worker)

---

3. Module index

ID	Module	Priority	Owner	Counterpart
M1	Request signing, replay protection, tenant secrets	P0	Habiba	Ehsan (client signers)
M2	Sessions, quotas, concurrency limits	P0	Habiba	Ehsan (worker release, SDK refresh)
M3	Worker capacity and provider failure	P1	Ehsan	none
M4	Telephony control plane	P0	Habiba	Ehsan (telephony SDK)
M5	Tenant isolation, database/RLS, admin	P0	Habiba	none
M6	Tool gateway, SSRF, prompt injection, agent powers	P1	Ehsan	Habiba (save-time checks)
M7	Secrets, PII, recordings, logging	P1	Habiba (server) + Ehsan (worker, laptop)	each other
M8	Resilience on Render free	P1	Habiba	Ehsan (SDK/demo timeouts)

M9	SDK developer integration	P1	Ehsan	none
M10	Supply chain, CI, release	P1	Ehsan	none
M11	Dashboard read-only requirement	P1	Habiba (enforcement) + Ehsan (UI, visibility)	each other
M12	Tests and observability	P2	Habiba (Python) + Ehsan (npm, worker)	none

---

4. Modules

Each module: **Scope**, **Known** (from the first read of the code), **Check** (what to verify), **Unread** (not looked at yet). "Check" items are questions, not findings.

M1. Request signing, replay protection, tenant secrets (P0, Habiba leads, Ehsan reviews the client signers)

Scope: `control_plane/mint.py`, `control_plane/secrets.py`, `secrets_db.py`, `secret_crypto.py`, `tenant_portal_api/machine_auth.py`, migrations `0004_nonces.sql`, `0009_tenant_secrets.sql`, `0033_tenant_secret_encryption.sql`, `scripts/encrypt_tenant_secrets.py`. Client signers (Ehsan reviews): `sdk-server/src/index.ts`, `telephony/src/signing.ts`, `client-deliverables-final/host-backend-starter/src/signing.js`, `client-integration-test/backend/src/signing.js`.

Known

- Two HMAC message shapes: mint `tenant_id.ts.nonce.agent_id` (`control_plane/mint.py:L40-L44`) and machine `tenant_id.ts.nonce.action.body_hash` (message built at `tenant_portal_api/machine_auth.py:L72-L76`, body hash at `L57-L69`). The machine path uses `hmac.compare_digest` (`machine_auth.py:L160`); the mint path's compare is not recorded.
- Replay window 60 s, mint JWT TTL 120 s (`mint.py:L27-L28`); machine auth imports the same window (`machine_auth.py:L170-L171`). Nonces are inserted into `used_nonces`, and a unique violation means replay (`mint.py:L138-L145`, `machine_auth.py:L173-L179`).
- Nonce rows are deleted only by `scripts/purge_used_nonces.py:L56-L57`, called from `.github/workflows/reconcile.yml:L110-L112`, not by the mint module.
- Secret sources: DB provider with a cache TTL (`CP_DB_SECRET_CACHE_TTL_SEC`, default 60 s) or env map `CP_TENANT_SECRETS` (`control_plane/app.py:L52-L53`, `secrets.py:L46-L47`, `secrets_db.py:L45`).
- Tenant and tool secrets appear to be encrypted at rest: `control_plane/secret_crypto.py`, migration `0033_tenant_secret_encryption.sql` and `scripts/encrypt_tenant_secrets.py` exist; the worker imports `decrypt_tool_secret` (`worker/config.py:L22`) and admin code calls into it (`admin/queries.py:L232`). **None of this has been read.**
- `host-tools` compares its shared secret with `!==` (`host-tools/src/createApp.js:L21-L28`).

Check

☐ All four signers and both servers build the identical message and body hash (canonical JSON, empty-body case, header names). Telephony has frozen test vectors (`telephony/test/phase9-contract.mjs`); no signing tests are recorded for the two JS starters.
☐ Mint-path signature compare is constant-time.
☐ Nonce insert is atomic under concurrent requests; behavior when the DB is unreachable (fails closed?).
☐ Purge age is longer than the replay window, and the purge workflow actually runs in production (not confirmed yet).
☐ How long an old secret keeps working after rotation (cache TTL); `CP_TENANT_SECRETS` is not used on hosted services.
☐ Secret encryption: algorithm, where the key lives, rotation, and which machines need it. The worker decrypts tool secrets, so the key (or a way to get it) must exist on the worker machine; confirm.
☐ Rate limits (mint: tenant 120/min, IP 240/min; machine: tenant 30/min, IP 120/min) are applied where intended. They are in-memory, so they reset on restart.

Unread: `control_plane/secrets_db.py` and `secret_crypto.py` bodies, most of `control_plane/app.py` (1,100 lines).

M2. Sessions, quotas, concurrency limits (P0, Habiba leads, Ehsan covers the worker and browser side)

Scope: `control_plane/app.py`, `control_plane/mint.py` (quota), `tenant_portal_api` session and `test_studio.py` routes, quota/session tables in `supabase/migrations/**`, `scripts/reconcile_sessions.py`. Worker side (Ehsan): `worker/session_close.py`, `worker/stale_jobs.py`, `worker/session_opening.py`, `scripts/drain_stale_agent_jobs.py`, `scripts/simulate_worker_crash.py`, `scripts/concurrency_test.py`. Browser side (Ehsan): `sdk/src/index.ts` refresh logic.

Known

- Mint checks concurrent and monthly quota gates (`mint.py:L3-L7`). On dispatch failure the control plane marks the session ended and decrements `concurrent_now` (`control_plane/app.py:L751-L766`).
- The control plane has no session-end endpoint: only `POST /v1/session`, `/v1/session/dev-mint` and `/v1/session/refresh` (`app.py:L957, L1045, L1084`). Normal teardown releases the slot from the worker (`worker/session_close.py:L72-L86`).
- Refresh hard cap `MAX_REFRESHES = 720` (`app.py:L479-L481`). The count is carried in the token's `refresh_count` metadata, **not** in process memory (`app.py:L484-L492`), so a restart does not reset it.
- Browser SDK: refresh at (expiresIn - 60) s, backoff 1 to 8 s, max 20 attempts, single-flight `refreshInFlight` (`sdk/src/index.ts:L564-L687`). No cap on SDK instances or listeners.

- No single constant for "max concurrent sessions per tenant" was found.
- Existing tools nobody has read yet: `scripts/concurrency_test.py`, `simulate_worker_crash.py`, `drain_stale_agent_jobs.py`, `reconcile_sessions.py`; Python `tests/**` (128 files).

Check

☐ What the real per-tenant and global concurrency limits are (DB defaults, migrations) and what a user sees when they hit them.
☐ A slot is never leaked: worker crash, laptop sleep, network loss. Read `simulate_worker_crash.py`, `drain_stale_agent_jobs.py`, `worker/stale_jobs.py` and `reconcile_sessions.py`: do they release `concurrent_now`, and do they actually run?
☐ Mint succeeds while no worker is registered or the worker is full: what does "dispatch failure" actually detect?
☐ The refresh cap is enforced on every refresh, and whether anything else bounds a call's total duration.
☐ Run `scripts/concurrency_test.py` (locally) before writing new load tests; record what it proves and what it doesn't.

Unread: most of `control_plane/app.py` handler bodies, `worker/stale_jobs.py`, the scripts above, quota SQL.

M3. Worker capacity and provider failure (P1, Ehsan)

Scope: `worker/main.py` (about 1,630 lines), `worker/providers/**` (21 files), `latency.py`, `provider_retries.py`, `provider_client_cache.py`, `stale_jobs.py`, `health_http.py`, `telephony_runtime.py`, `telephony_tts.py`, `humanization/**` (6 files), `scripts/dev_worker.py`.

Known

- Entry: `cli.run_app(WorkerOptions(...))` with idle processes default 3 (`LIVEKIT_NUM_IDLE_PROCESSES`) and init timeout 60 s (`worker/main.py:L1616-L1628`). Only these two have been seen so far; no load threshold or job memory limit was found.
- Provider connect retries default max 1, interval 1 s, timeout 30 s; env caps retries at 5 (`worker/provider_retries.py:L17-L19, L83-L106`).
- Registry supports STT gladia/deepgram, LLM gemini/groq, TTS uplift/cartesia/elevenlabs/rime; an unknown provider raises `UnsupportedProviderError` with no silent fallback (`worker/providers/registry.py:L11-L12, L32-L80`).
- Soniox STT exists only in the legacy factory (`worker/factories.py:L28-L40`); Fish Audio TTS has an adapter but is not in the registry (`registry.py:L58-L80`).
- Tool HTTP timeouts connect 1 s, read 4 s, write 2 s, pool 1 s; 16 connections (`worker/tools.py:L32-L33`).

Check

☐ Measure how many concurrent calls this machine really handles with real providers (CPU, memory, p95 turn latency). Find what happens at the load threshold. Is it configured or left at LiveKit's default?
☐ Laptop sleep, Wi-Fi drop, process kill/SIGTERM: what happens to live calls, drain behavior, quota release (links to M2).
☐ For each STT/LLM/TTS vendor: timeout, 429, mid-call disconnect, and what the caller hears.
☐ Legacy paths (Soniox factory, Fish adapter): reachable or dead?
☐ Which LLM model is actually used when the agent doesn't specify one (SDK default vs worker default; see M9).
☐ Empty/log-only `except` blocks in `worker/main.py` and across the 63 modules.

Unread: most of the 63 worker modules, provider caches, per-turn maps, codec/sample-rate settings in the adapters.

M4. Telephony control plane (P0, Habiba leads, Ehsan reviews the SDK)

Scope: `tenant_portal_api/telephony_service.py` (2,477 lines, unread), `telnyx_client.py` (1,084), `livekit_sip.py` (571), `telephony_queries.py` (533), `telephony_credentials.py` (220), `telephony_errors.py` (136), `telephony_reconcile.py` (97), `telephony_routes.py`, `telephony_webhooks.py`, `telnyx_destinations.py`, `telephony_config.py`, `telephony_health.py`; migrations `0012` to `0015` and the two `20260801*` telephony migrations; `scripts/reconcile_telephony.py` and the Telnyx `*.mjs` helpers. Client (B): `telephony/src/**` (7 files).

Known

- Webhook route `/webhooks/telephony/telnyx` verifies Ed25519 over `{timestamp}|{raw_body}`; body cap 256 KiB; replay window 300 s; IP rate 600/min before verification (`tenant_portal_api/telephony_webhooks.py:L35-L36, L40, L52-L68, L143-L190, L363-L369`).
- De-duplication of signatures and event ids is an in-memory LRU of 20,000 entries (limit at `telephony_webhooks.py:L43-L45`, trim helper `L71-L75`), so it resets on restart.
- In mock mode, verification accepts when no public key is set (`telephony_webhooks.py:L159-L160`). `TELEPHONY_PROVIDER_MODE` defaults to `real` (`telephony_config.py:L14`).
- Outbound destinations: `TELNYX_OUTBOUND_DESTINATIONS` (unset gives US/CA-only behavior; list or `all`) at `telnyx_destinations.py:L162`. Where it is enforced was not read.
- Tenant Telnyx credentials are encrypted with `TELEPHONY_CREDENTIAL_ENCRYPTION_KEY` (`telephony_credentials.py:L160, L179`); implementation not read.
- Migration file names point to database-level idempotency and tenant-scoped webhooks (`0013_telephony_constraints_indexes_status_idempotency.sql`, `0014_telephony_idempotency_webhook_tenant_scope.sql`), plus `0014_telephony_data_governance_audit.sql` and `0015_telephony_rls_grants.sql`. **File names only; contents unread.**
- 28 frozen operations in the SDK (`telephony/src/routes.ts:L3-L32`); SDK has no timeout or retry.

Check

☐ Idempotency across restarts: read migrations `0013` and `0014_*idempotency*` and `telephony_service.py` to see whether webhook handling is idempotent in the **database**, not just the LRU. What does `telephony_reconcile.py` repair?

☐ A hosted deployment can never run in mock mode or accept webhooks without a public key.
☐ Inbound webhook arriving while a Render service is asleep: what is lost and how it recovers (links to M8).
☐ Outbound spend and abuse controls: where the destination list is enforced, whether any of the 28 operations can bypass it, and whether per-tenant call caps, rate limits and concurrent-call caps exist.
☐ Credential encryption: algorithm, key handling and rotation, never logged or returned.
☐ Per-tenant isolation of SIP/LiveKit resources; call state machine (`telephony_webhooks.py:L123-L140`).
☐ Timeouts and retries on Telnyx and LiveKit HTTP clients (unread).

Unread: everything listed in Scope except `telephony_webhooks.py`, `machine_auth.py` and the config slices.

M5. Tenant isolation, database/RLS, admin (P0, Habiba)

Scope: `tenant_portal_api/{app.py, queries.py, membership.py, supabase_auth.py, supabase_admin.py, portal_access.py, session_cookie.py, jwt_secret.py}`, the agent-ownership and origin checks in `control_plane/mint.py`, `admin/**` (6 files), `supabase/migrations/**` (39 files). Start with `0002_rls.sql`, `0015_telephony_rls_grants.sql`, `0031_login_throttle_and_totp_replay.sql`, `0032_admin_disable_and_revocation.sql`, `0034_tenant_members.sql`, `0035_tenant_members_status.sql`, `0030_agents_archive.sql`, and `scripts/rls_check.py`, `scripts/verify_admin_boundary_live.py`.

Known

- Mint checks the agent belongs to the tenant and enforces an origin allowlist (empty hosted list fails closed) (`control_plane/mint.py:L151-L178`). Machine auth loads tenant status and secret by `tenant_id` (`machine_auth.py:L144-L161`).
- Services connect with `SUPABASE_DB_URL` (psycopg); the portal and worker also use the service-role key for storage/admin (`supabase_admin.py:L17-L24`, `worker/session_recording.py:L29-L30`); the browser uses the anon key.
- Limits: 100 agents per tenant (`PORTAL_MAX_AGENTS_PER_TENANT`), prompt 24,000 chars, greeting 2,000, page limit 200 (`tenant_portal_api/app.py:L150-L161`).
- `TENANT_PORTAL_JWT_SECRET`: the variable name is at `jwt_secret.py:L32`; local generation when missing and failure when hosted are at `L45-L62`. Flags `PORTAL_ALLOW_TENANT_BOOTSTRAP`, `PORTAL_ALLOW_BROWSER_SECRET_REVEAL`, `PORTAL_ALLOW_BROWSER_SECRET_ROTATE` exist (`portal_access.py:L31`, `session_cookie.py:L35, L49`).
- Admin: PBKDF2 passwords + TOTP (`admin/security.py:L30-L63`); login throttle 8 per identity, 30 per IP, 15 min (`control_plane/login_guard.py:L23-L25`, `admin/app.py:L240-L276`); the rotate route accepts any `tenant_id` under admin auth and returns the new secret once (`admin/app.py:L366-L387`).

Check

☐ Every portal query is scoped by the tenant from the verified token, not from request input (`queries.py`, 903 lines, unread).
☐ RLS policies: which tables the anon/authenticated roles can read or write, and which database role each service connects as (a privileged role bypasses RLS). Run `scripts/rls_check.py` first and read what it covers.
☐ "Hosted" is detected correctly on every Render service (`UVA_ENV` / `RENDER` set), so secrets are never auto-generated into `.env.local` in production. Defaults and hosted values of the three `PORTAL_ALLOW_*` flags.
☐ The limits above are enforced on the machine (HMAC) agent routes too, not only on the portal route.
☐ Admin: PBKDF2 iteration count, TOTP window and replay protection (migration 0031), disable/revocation (0032), and that admin actions are audited.
☐ Tenant members and roles (`membership.py`, 0034/0035): who can see or do what inside a tenant.

Unread: `supabase/migrations/**`, `queries.py`, `membership.py` (475), most of `app.py` (1,262), `admin/queries.py` (316).

M6. Tool gateway, SSRF, prompt injection, agent powers (P1, Ehsan leads, Habiba reviews save-time checks)

Scope: `worker/tools.py`, `worker/ssrf_guard.py`, `control_plane/runtime_env.py` (Habiba's file, used by the guard), `host-tools/**` (9 files), `tenant_portal_api/tools_webhook.py` (Habiba, 189 lines, unread), prompt assembly in `worker/main.py`, migrations `0008_tools.sql`, `0026_agents_tools_webhook.sql`.

Known

- Worker tool calls POST to `{base}{path}` through `prepare_tools_post_url` and include `tenant_id/agent_id` (`worker/tools.py:L236-L274`). Headers are built at `L164-L167`; the call is refused when no gateway secret is available (`L249-L255`).
- The SSRF guard has "hosted checks" and blocks private/metadata names (`worker/ssrf_guard.py:L1-L5, L19-L34`). It imports `control_plane.runtime_env.is_hosted` (`ssrf_guard.py:L15`).
- Tool failures return `{"error": ..., "success": False}` (`worker/tools.py:L291-L292`).
- `host-tools` (demo gateway): `tenant_id` comes from the request body (`createApp.js:L81`), a debug endpoint lists appointments with names and phones (`L214-L217`), 500s return `err.message` (`L244-L252`), in-memory maps never clean up completed idempotency keys or cancelled rows (`store.js:L72-L83`).
- Groq completion cap default 96 tokens (`worker/providers/llm/groq.py:L41`).

Check

☐ **Read `control_plane/runtime_env.py`: what does `is_hosted` look at, and what does it return on the local worker?** Then list which SSRF checks switch off when it returns false. Also redirects, DNS rebinding, IPv6, metadata hostnames.
☐ Save-time validation of `toolsBaseUrl` in the portal (`tools_webhook.py`) matches the request-time guard.
☐ Which tools are write-capable (book/reschedule/cancel), whether they need confirmation, and how user speech reaches tool

arguments (injection).
☐ Limits on tool loops, token use and call duration (unbounded consumption).
☐ Is `host-tools` only a demo, or does any customer run it as-is?

Unread: `tools_webhook.py`, `runtime_env.py`, how `toolsAuthSecret` is stored and returned, prompt-assembly code beyond `main.py:L1-L7`.

M7. Secrets, PII, recordings, logging (P1, Habiba for the server side, Ehsan for the worker and laptop)

Scope: environment variables read in code (`.env.example`, `os.getenv` / `process.env` reads); `worker/{transcript_logging,prompt_dump,recording_policy,session_recording,session_retention}.py`; `tenant_portal_api/{recording_urls,supabase_admin,telephony_credentials}.py`; Sentry setup in `control_plane/app.py`; credentials page in the dashboard; migrations `0027_session_call_recordings.sql`, `0028_session_recording_consent_retention.sql`, `0014_telephony_data_governance_audit.sql`; `scripts/purge_expired_session_media.py`.

Known

- The worker needs `LIVEKIT_API_KEY/SECRET`, `SUPABASE_DB_URL`, `SUPABASE_SERVICE_ROLE` and provider keys; it loads `.env.local` (`worker/main.py:L1575-L1629`). It runs on a local machine. It also imports secret-decryption code from the control plane (`worker/config.py:L22`).
- `UVA_LOG_TRANSCRIPTS` default off, text capped at 200 chars when on (`worker/transcript_logging.py:L17-L22`); `UVA_DUMP_PROMPTS` default off (`worker/prompt_dump.py:L22`); `UVA_SESSION_RECORD_AUDIO` default `0` (`worker/recording_policy.py:L30`); retention days default is a code constant (`worker/session_retention.py:L26`). Recording policy checks `RENDER` only (`recording_policy.py:L40`).
- Control plane has optional Sentry with `traces_sample_rate=0.1` (`control_plane/app.py:L130-L137`).
- The credentials page shows `hmac_secret` to owners (`dashboard/src/app/credentials/page.tsx:L147-L153, L192-L206, L322-L346`).
- Telephony SDK redacts responses and errors (`telephony/src/errors.ts:L111-L114`, `transport.ts:L15-L29`); the agents SDK sends `toolsAuthSecret` in create/update bodies and has no equivalent redaction layer (`sdk-server/src/index.ts:L80-L81`).
- The starter returns the control-plane URL in its 502 detail (`host-backend-starter/src/createApp.js:L118-L124`).

Check

☐ **Laptop:** disk encryption, who else can log in, `.env.local` never committed, secrets scope (can the worker use a narrower DB role instead of the service role?), where the secret-decryption key lives.
☐ Where prompt dumps and transcript logs go when switched on; whether Sentry scrubs request bodies.
☐ Recording storage, signed URL lifetime, retention actually enforced (`purge-session-media.yml`, migration 0028).
☐ PII at rest: telephony call rows (phone numbers), transcripts in DB, host-tools memory.
☐ Error messages and logs never contain secrets or internal URLs.

Unread: `recording_urls.py` (86), `telephony_service.py` call rows, most of `worker/**`.

M8. Resilience on Render free (P1, Habiba leads, Ehsan covers SDK and demo timeouts)

Scope: `control_plane/**`, `tenant_portal_api/**`, `admin/**`, the SDK timeouts, `host-tools`, demos/starters.

Known

- In-memory state: rate buckets (`control_plane/app.py:L406-L435`, `machine_auth.py:L37-L44, L79-L87`), webhook LRUs (`telephony_webhooks.py:L43-L45`), demo caches `_capsCache` and `_pipelineAppliedByAgent` (`client-integration-test/backend/src/createApp.js:L90-L108, L251`).
- Voice SDK fetch timeout default 15 s and LiveKit connect timeouts use the same value (`sdk/src/internal/http.ts:L8-L27`, `sdk/src/index.ts:L303-L306`). Agents/telephony SDKs: none. Starter and demo backend: no upstream timeout (no `AbortController` found).
- Agents SDK calls `JSON.parse` on the response with no try/catch (`sdk-server/src/index.ts:L269`); the telephony SDK maps invalid JSON to an error code (`telephony/src/errors.ts:L42-L47`).
- Admin DB connect timeout 10 s (`admin/app.py:L153`); control-plane mint runs sync DB calls in a threadpool (`control_plane/app.py:L1-L8`).

Check

☐ List exactly which services run on Render and their plan limits (RAM/CPU), from your dashboard.
☐ First request after idle: what each client sees (SDK timeout, non-JSON error page, 502) and whether that is a clear, retryable error.
☐ For every in-memory structure above, what breaks on restart and whether the database covers it.
☐ Health endpoints and whether anything keeps services awake. Keep-alive pings across several services can use up the shared monthly hours.
☐ DB connection handling under restarts (pooling, timeouts) on a small instance.

M9. SDK developer integration (P1, Ehsan)

Scope: `sdk/`, `sdk-server/`, `telephony/`, `client-deliverables-final/**` (14 files), `client-integration-test/**` (24 files), `dashboard/src/content/docs/**` (16 files), `docs/HOST_BACKEND_CONTRACT.md`.

Known

- Docs vs code: voice README omits `sessionHeaders`, `sessionCredentials`, `connectTiming` (`sdk/README.md:L62-L82`); `ConnectOptions.voiceId` is typed but never sent (`sdk/src/index.ts:L52-L55, L275-L278`); README points to a contract doc that says it is superseded (`docs/HOST_BACKEND_CONTRACT.md:L1-L12`); telephony README and CHANGELOG claim npm provenance but the workflow omits `--provenance` (`release-sdk.yml:L155-L156`); the dashboard lifecycle sketch omits the `connect_timing` step.
- Packaging: all three packages are ESM-only with no `require` condition; no `license` field and no LICENSE file in any of them; no `engines` on voice and agents (`sdk/package.json:L25-L38`).
- Agents SDK: plain `Error` from the constructor, unguarded `JSON.parse`, no timeout, no retry. Portal machine routes allow 30 requests/min per tenant (`machine_auth.py:L37-L39`).
- Defaults differ: agents SDK sends `gemini-2.5-flash` when `llmModel` is omitted (`sdk-server/src/index.ts:L196`); worker default is `gemini-3.6-flash` with remapping of deprecated ids (`worker/providers/llm/gemini.py:L30`).
- Test gaps: no `connect()` happy-path, `disconnect`, getters test for voice; agents has no tests for `getProviderCapabilities`, `listManagedNumbers`, `assignAgentToNumber`; telephony covers all 28 operations.
- No `examples/` folder. `host-backend-starter` does not use the SDKs. The demo backend reflects any `Origin` when the allowlist is empty and uses `express.json()` with no limit (`createApp.js:L54-L59, L262`); the starter does neither (`createApp.js:L44-L53, L63`).

Check

☐ Voice SDK: is the refresh URL from the host response validated before the session token is sent to it (`sdk/src/index.ts:L690-L710`)? *Open question only; nothing found yet.*
☐ Load the agents/telephony packages from a **CommonJS** backend (e.g. NestJS) and see whether it works. ESM-only may block common setups.
☐ Follow the README quickstart literally, voice, agents and telephony, and write down every place it fails or misleads.
☐ Decide the right fix for each docs/code mismatch above (doc change or code change).
☐ Is `client-deliverables-final/` shipped to outside clients? If yes, audit it as product, not as demo.

M10. Supply chain, CI, release (P1, Ehsan)

Scope: `.github/workflows/*` (7 files), all lockfiles (`pnpm-lock.yaml`, `sdk/`, `sdk-server/`, `telephony/`, `dashboard/` `package-lock.json`), root `package.json`, `requirements.txt`, `docker/*.Dockerfile` (4) and `docker/requirements-*.txt` (3), `Makefile`.

Known

- CI `security-scan` runs pip-audit/npm audit with `continue-on-error`, plus gitleaks and a secret-shape grep over `sdk/dist` (`.github/workflows/ci.yml:L152-L206`). `pytest-integration` exits 0 and skips when `SUPABASE_DB_URL` is unset (`ci.yml:L40-L70`). `make test` runs pytest only (`Makefile:L9-L12`).
- Release publishes with `npm publish --access public` and no `--provenance` (`release-sdk.yml:L120-L156`). How the workflow authenticates to npm is not recorded.
- No deprecated, git or `*` runtime dependencies found in the three SDK lockfiles; voice lockfile has optional `fsevents` with an install script and `*` peer ranges on transitive optional peers (`sdk/package-lock.json:L332, L561-L567, L1125`). Dashboard lockfile not fully scanned.
- Scheduled workflows touch the production database (`reconcile.yml`, `purge-session-media.yml`).

Check

☐ How npm publishing authenticates today. npm permanently revoked classic tokens on 2025-12-09, and trusted publishing (OIDC) is the replacement. Provenance needs a public repo; the workflow comment cites a private repo and the publish step omits `--provenance`, so the "provenance" claims in the telephony README/CHANGELOG don't match what is published.
☐ Whether audit findings can ever fail the build, and whether integration tests ever run in CI.
☐ Production secrets in workflows: scoping, environments, who can trigger `deploy-prod`.
☐ Python pins in `requirements.txt` and `docker/requirements-*.txt` (loose vs exact), Docker base images, dashboard lockfile scan.

Unread: `deploy-prod.yml` (94 lines), `deploy-staging.yml` (84), `reconcile.yml` (125), `purge-session-media.yml` (105), `refresh-voice-previews.yml` (35), Dockerfile bodies beyond CMD.

M11. Dashboard read-only requirement (P1, Habiba for enforcement, Ehsan for UI and visibility)

> **Decision needed from Habiba:** the stated rule is "users can read; users cannot create agents in the dashboard." Are **edits** (prompt changes), **credential reveal/rotate**, **telephony actions** and **Test Studio** sessions also off-limits? The audit records the current state either way.

Scope: `dashboard/**` (Ehsan); portal routes the dashboard calls, `tenant_portal_api/app.py` (Habiba); RLS and grants on tables that hold agents (`supabase/migrations/**`: `0002_rls.sql`, `0030_agents_archive.sql`, `scripts/rls_check.py`) (Habiba).

Known

- Dashboard has a Supabase browser client with `persistSession` and `autoRefreshToken` (`dashboard/src/lib/supabaseBrowser.ts:L19-L25`). The portal token is held in memory, not localStorage (`dashboard/src/lib/portalAuth.ts:L3-L15`). Overview data is cached in sessionStorage for 15 minutes (`overviewCache.ts:L3-L19`); SWR does not revalidate on focus (`SwrProvider.tsx:L11-L19`).
- The portal defines `CreateAgentBody` and enforces `MAX_AGENTS_PER_TENANT` (`tenant_portal_api/app.py:L150-L161`). Earlier notes say the agent prompt is edited via portal APIs from the UI, but the dashboard page bodies have not been read.
- Telephony UI calls the portal with `fetch` (`dashboard/src/lib/telephonyApi.ts:L44`); credentials UI can show `hmac_secret`; portal

flags allow browser secret reveal/rotate (`session_cookie.py:L35, L49`).
- Agents created in code go through `/machine/agents` (HMAC); how they reach the dashboard's list has not been traced.

Check (visibility: Ehsan + Habiba)

☐ An agent created with the agents SDK appears in the dashboard: trace machine route to table to portal list route to page. Same tenant filter, no field loss, archived agents (`0030_agents_archive.sql`) handled as intended.
☐ How stale it can be (15-minute overview cache, SWR dedupe) and whether a new agent is hidden for that long.

Check (enforcement: Habiba + Ehsan)

☐ Which portal routes can create or change agents with a dashboard login (JWT), and whether each is intended.
☐ Whether a signed-in user can write agent rows straight through the Supabase browser client (RLS review).
☐ UI has no create-agent path (Ehsan checks), and the server rejects it even if someone calls the API directly (Habiba checks).
☐ Once the decision above is made, repeat for the other actions.
☐ General dashboard hygiene: `NEXT_PUBLIC_*` only, XSS surfaces, caches in sessionStorage.

Unread: nearly all of `dashboard/src/app/**` and components, `supabase/migrations/**`.

M12. Tests and observability (P2, Habiba for Python, Ehsan for npm and worker)

Scope: `tests/**` (128 files, split by code owner), npm test folders (`sdk/src/**/*.test.ts`, `sdk-server/test/`, `telephony/test/`, `host-tools` test), worker logging and health modules (`worker/latency.py`, `health_http.py`, `transcript_logging.py`), `scripts/run_ci_unit.py`.

Known: no load, soak, concurrency or failure-injection suites in any npm package. Python has `scripts/concurrency_test.py` and `simulate_worker_crash.py` (unread) and 128 files under `tests/`. Sentry exists only in the control plane and is optional; no Prometheus/OTel in the portal or control plane; no correlation ids in the SDKs or demos. Worker has loggers, an optional health port (`UVA_WORKER_HEALTH_PORT`, off by default) and opt-in turn-latency publishing.

Check

☐ Inventory Python tests: what they cover for M1 to M5 and what is missing.
☐ Add the smallest useful concurrency and failure tests where M2/M3 find gaps.
☐ Decide the minimum logging fields (tenant id, session id) so a failed call can be traced across services.

---

5. Shared code between Habiba and Ehsan (do not change alone)

1. **HMAC shapes and `used_nonces` (M1).** Habiba owns the server; Ehsan owns the client signers. Any change needs both of you.
2. **`tenant_portal_api` imports `worker.providers`** (`provider_capabilities.py:L22`, `provider_validation.py:L23-L38`). Ehsan: do not change `worker/providers/registry.py`'s public interface without telling Habiba.
3. **Portal imports `worker.humanization.history.sanitize_transcript_turns`** (`tenant_portal_api/queries.py:L22`, `telephony_service.py:L2472`). Ehsan should not rename or change that function without telling Habiba.
4. **The worker imports Habiba's code**, in three places: `control_plane.runtime_env.is_hosted` (`worker/ssrf_guard.py:L15`), `control_plane.secret_crypto.decrypt_tool_secret` (`worker/config.py:L22`) and `tenant_portal_api.telephony_config.is_mock_provider_mode` (`worker/telephony_runtime.py:L16`). Habiba should not change these without telling Ehsan. The first one decides SSRF behavior on the local worker (M6).
5. **Quota lifecycle (M2/M3).** Mint and rollback are in the control plane; release is in the worker. Change one side only after checking the other.
6. **Tool URL and secret (M6).** Save-time checks (portal, Habiba) and request-time checks (worker, Ehsan) should end up equivalent.
7. **Recordings (M7).** Worker writes to storage; portal signs URLs.
8. **Dashboard read-only (M11).** UI changes (Ehsan) and server/RLS changes (Habiba) ship together.
9. **Shared files:** see the ownership rule in Section 2 (Habiba: database and migrations; Ehsan: build and release files).

---

6. Where to start

- **Habiba:** read `supabase/migrations/**` (start with the files named in M5), then `telephony_service.py` and `queries.py`. M4, M5 and M11 can't be judged without them. Run `scripts/rls_check.py`.
- **Ehsan:** read `worker/main.py`, `worker/ssrf_guard.py` and `control_plane/runtime_env.py`, run the README quickstarts for M9, and run `scripts/concurrency_test.py` locally.

---

7. Not read yet (read before judging)

Line counts are physical lines.

- **Habiba's code (Python):** `tenant_portal_api/telephony_service.py` (2,477), `telnyx_client.py` (1,084), `queries.py` (903), `livekit_sip.py` (571), `telephony_queries.py` (533), `membership.py` (475), `provider_validation.py` (326), `test_studio.py` (230), `telephony_cr`

edentials.py (220), tools_webhook.py (189), telephony_errors.py (136), telephony_reconcile.py (97), recording_urls.py (86), most of app.py (1,262); control_plane/app.py (1,100) beyond slices, secrets_db.py (157), warm.py (62), runtime_env.py, secret_crypto.py; admin/queries.py (316) and other admin routes.

- **Database:** supabase/migrations/** (39 files: tables, RLS, indexes, grants).
- **Ehsan's code:** most of worker/** (63 modules, 10,476 lines); dashboard/src/app/** and components; client-integration-test/frontend/src/main.ts (1,187 lines, read to line 150) and telephonyPanel.ts (399); demo backend createApp.js (516, read to about line 370) and telephonyRoutes.js (303); starter config.js (35) and server.js (11).
- **Ops:** .github/workflows/: deploy-prod.yml (94), deploy-staging.yml (84), reconcile.yml (125), purge-session-media.yml (105), refresh-voice-previews.yml (35); scripts/** (50 files); Dockerfile bodies; docs/**; Python tests/** (128 files).
- **Not assessed at all:** self-serve-demo-UI/ (has its own npm packages), voice-picker/, state/.

---

8. Open questions

1. Which services run on Render today, and is any database on Render (vs Supabase)?
2. Expected peak concurrent calls (per tenant and overall)? This sets the load-test target.
3. The worker machine: which machine, always on, who restarts it, any second worker planned?
4. Is the nonce purge workflow actually scheduled in production?
5. Dashboard read-only scope beyond "no agent creation" (see M11).
6. Is TELEPHONY_PROVIDER_MODE=real set on every hosted service, and is mock mode used anywhere outside tests?
7. Will the repo stay private (affects provenance claims, M10)?
8. Does client-deliverables-final/ go to outside clients as-is (changes the weight of M9)?
9. Are self-serve-demo-UI/, voice-picker/ and state/ deployed or customer-facing? If yes, they need a module.

---

9. How to record findings

Each of you keeps your own file (FINDINGS_HABIBA.md, FINDINGS_EHSAN.md), one block per finding:

```
ID:           M4-F01
Module:       M4
Severity:     Critical | High | Medium | Low | Info
Where:        path:line  (opened by you, not copied from this plan)
What:         one or two sentences
Impact:       who is affected and when
How verified: read code | ran test | ran locally  (say which)
Suggested fix:
Status:       open | in progress | fixed | won't fix (reason)
```

Severity: **Critical** = cross-tenant access, auth bypass or uncontrolled spend. **High** = data exposure, or outage under realistic load. **Medium** = wrong behavior with a workaround. **Low** = hygiene. **Info** = worth knowing.

Working rules:

1. Audit first, fix second. Fixes go through the module's lead; shared contracts (Section 5) need both of you.
2. Re-open the file before filing; file:line references here are only pointers.
3. No dashboard behavior changes until the M11 decision is made.
4. Run load tests locally or on staging, not on production or free-plan services you depend on.
5. "NOT FOUND" means you searched and say what you searched for.